

Acoustic Side-Channel Attack on Mechanical Keyboards

Software-Based Keystroke Reconstruction Using Frequency Signatures

Cyber-Physical Systems Project

By

23MEI10066 Dev Soni
23MEI10067 Shreyansh Tripathi
23MEI10069 Himalaya Yadav
23MEI10070 Eklavya Mathur
23MEI10085 Akshit Agrawal

Course Instructor

Dr. Hemraj S Lamkuche
School of Computing Science Engineering and Artificial Intelligence
VIT Bhopal University, Sehore
Madhya Pradesh

September 8, 2026

1 General Application

This project implements a *software-based simulation* of an acoustic side-channel attack on a mechanical keyboard. In the proposed approach, each keyboard key is associated with a custom frequency signature. When a key is pressed within the software model, its corresponding frequency is generated and represented as a synthetic signal.

The software then analyzes the frequency pattern and maps it back to the corresponding key. The fundamental pipeline is:

**Key Press → Custom Frequency → Frequency Analysis → Key Identification →
Keystroke Reconstruction**

This approach demonstrates the acoustic side-channel concept entirely through software, without requiring a microphone or additional sensing hardware. It is a *prototype/simulation* of the side-channel reconstruction concept and the corresponding Cyber-Physical Systems (CPS) verification requirements.

2 Objective

The main objective is to develop a software system capable of identifying and reconstructing keystrokes using predefined frequency signatures. The system aims to:

1. Assign a distinct frequency/signature to each key.
2. Generate or process the corresponding frequency when a key is activated.

3. Analyze the frequency signal.
4. Match the detected frequency with the corresponding key.
5. Reconstruct the sequence of detected keystrokes.
6. Measure the execution time and accuracy of the system.
7. Verify that the system maintains predefined safety and correctness properties.

3 Hardware and Software

3.1 Hardware

Since the implementation is software-only, the hardware requirements are minimal:

- A computer or laptop.
- No dedicated microphone.
- No additional sensors.
- No modification to the physical keyboard.

3.2 Software

The software components include:

- The custom application (`acoustic_side_channel.py`).
- Signal generation/processing module.
- Frequency-matching/classification module.
- Keystroke reconstruction module.
- Waveform visualization module (`waveform_visualization.py`).
- Data storage/logging module.

4 Cyber-Physical Systems Application: WCET

The software operates as a real-time task. The processing pipeline is:

Input Key $\rightarrow$ Frequency Generation $\rightarrow$ Frequency Analysis $\rightarrow$ Frequency Matching $\rightarrow$ Key Output

The total execution time can be expressed as:

$$T_{\text{total}} = T_{\text{input}} + T_{\text{frequency}} + T_{\text{analysis}} + T_{\text{matching}} + T_{\text{output}}$$

We measure three timing quantities:

- **Average execution time:** the mean processing time per key.
- **Maximum execution time:** the observed worst case in a single run.
- **WCET (median):** the median of the per-key worst-case times across multiple trials.

- **WCET (P95)**: the 95th percentile of the per-key worst-case times across multiple trials.

Because a single-trial timing measurement can be dominated by scheduler or measurement noise, the implementation runs the reconstruction over a number of trials (`WCET_TRIALS` = 20) and reports *robust* statistics: the **median** (typical worst case) and the **P95** (a conservative worst case) of the observed reconstruction times. The P95 value is computed with a nearest-rank percentile,

$$\text{P95 index} = \min(\lceil n \cdot 0.95 \rceil - 1, n - 1),$$

which reports an exact 95th percentile for any trial count.

The real-time requirement is:

$$T_{\text{WCET}} < T_{\text{deadline}}$$

This ensures the software can process a key/frequency before the next required input is processed.

4.1 Why this matters

If the processing time becomes too large, the system may:

- Miss a key event.
- Process keys in the wrong order.
- Produce an incorrect reconstruction.
- Lose synchronization with the input sequence.

In our implementation, the deadline is set to 50 ms. The system always completes processing well within this bound, and the reported worst case (median and P95) remains below the deadline, satisfying the real-time requirement.

5 Finding and Proving Invariants

An *invariant* is a property that must remain true throughout program execution. Our system maintains the following invariants.

5.1 Invariant 1: Valid Frequency

Every generated/detected frequency must belong to the predefined frequency set:

$$F \in \{F_1, F_2, F_3, \dots, F_n\}$$

5.2 Invariant 2: Valid Key Mapping

Every valid frequency must map to a valid key:

$$F_i \rightarrow K_i$$

5.3 Invariant 3: One-to-One Mapping

Each frequency must correspond to exactly one predefined key. This prevents ambiguity between keys:

$$F_i \neq F_j \quad \text{for } i \neq j$$

5.4 Invariant 4: Sequence Preservation

If the input sequence is $K_1, K_2, K_3, \dots, K_n$, then the reconstructed sequence must preserve the same ordering (assuming all inputs are successfully processed).

6 Convergence, Liveness and Termination

6.1 Convergence

The system uses a frequency-matching threshold. The detected frequency should converge toward the predefined frequency/signature of the selected key:

$$|F_{\text{detected}} - F_{\text{key}}| < \epsilon$$

where ϵ is the permitted frequency error (the tolerance). When this condition is satisfied, the software identifies the corresponding key.

6.2 Liveness

The system must continuously be able to process new key/frequency events:

$$\text{Input} \rightarrow \text{Analyze} \rightarrow \text{Identify} \rightarrow \text{Output} \rightarrow \text{Wait for next input}$$

The system does not get permanently stuck during frequency analysis or key identification. This is verified programmatically by the `liveness_test()` function.

6.3 Termination

When the user stops the application:

$$\text{Stop} \rightarrow \text{Finish current operation} \rightarrow \text{Save results} \rightarrow \text{Release resources} \rightarrow \text{Exit}$$

A finite input sequence always terminates. This is verified by the `termination_test()` function, which checks that a fixed sequence completes within a bounded time.

7 Ranking Function

For this project, we define an error-based ranking function based on the difference between the detected frequency and the expected frequency:

$$V = |F_{\text{detected}} - F_{\text{expected}}|$$

A successful frequency-matching process minimizes this error. The desired condition is:

$$V \rightarrow 0$$

or, with a tolerance:

$$V < \epsilon$$

The frequency-error function V is used as a *ranking measure* for the identification process. As the detected frequency becomes closer to the predefined frequency of a key, the error decreases and the system approaches the correct key classification.

The key with the smallest V value (i.e., the minimum error) is selected as the matching key, provided $V \leq \epsilon$ (the tolerance).

8 System Will Never Do Something Catastrophic

For our software CPS, we define the *catastrophic/failure conditions* as:

- An invalid frequency being accepted as a valid key.
- Two keys having the same frequency.
- An incorrect key mapping.
- Processing exceeding the deadline.
- Loss of input sequence.
- The application entering an invalid state.

The safety conditions are therefore:

$$\begin{aligned} F &\in F_{\text{valid}} \\ F_i &\neq F_j \quad (i \neq j) \\ T_{\text{WCET}} &< T_{\text{deadline}} \\ \text{Output} &\in K_{\text{valid}} \cup \{\text{Unknown}\} \end{aligned}$$

The **Unknown** state is important: the software must not force an incorrect key when the frequency does not sufficiently match any predefined signature.

9 Waveform Visualization

To illustrate the acoustic concept visually, the application generates two types of plots using `matplotlib` and `numpy`.

9.1 Sine Waveform

Each key is associated with a sine wave:

$$x(t) = \sin(2\pi f_i t)$$

where f_i is the key's assigned frequency. Figure 1 shows the sine waves for the sequence HELLO WORLD.

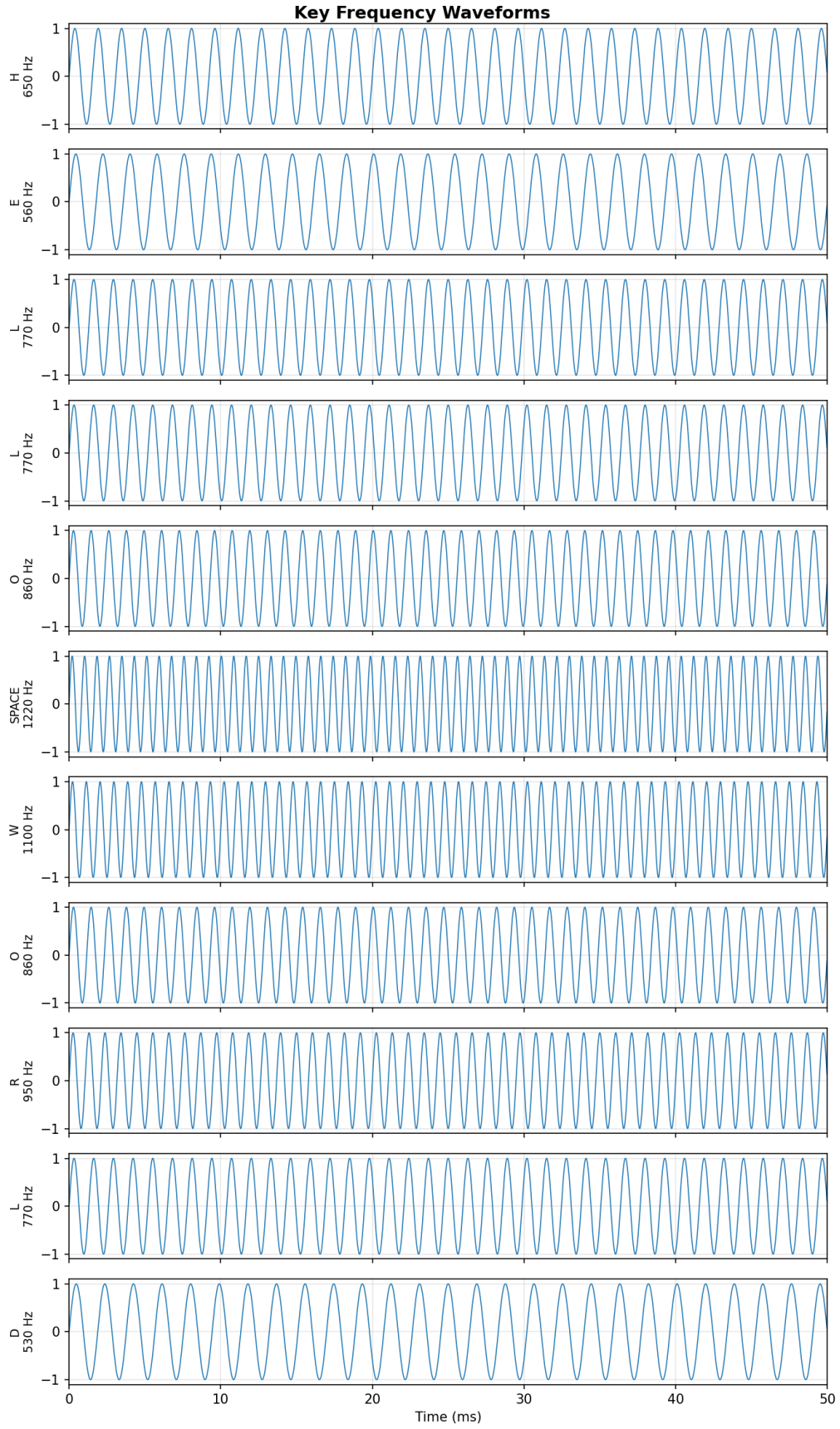


Figure 1: Sine waveforms for the key sequence HELLO WORLD, showing each key's assigned frequency.

9.2 Spectrogram

The spectrogram visualization (Figure 2) shows the frequency (Hz) on the vertical axis and time on the horizontal axis. Each key appears as a horizontal band at its assigned frequency. The displayed frequency range spans 350–1900 Hz, which covers all 48 key signatures, including the digits and punctuation marks.

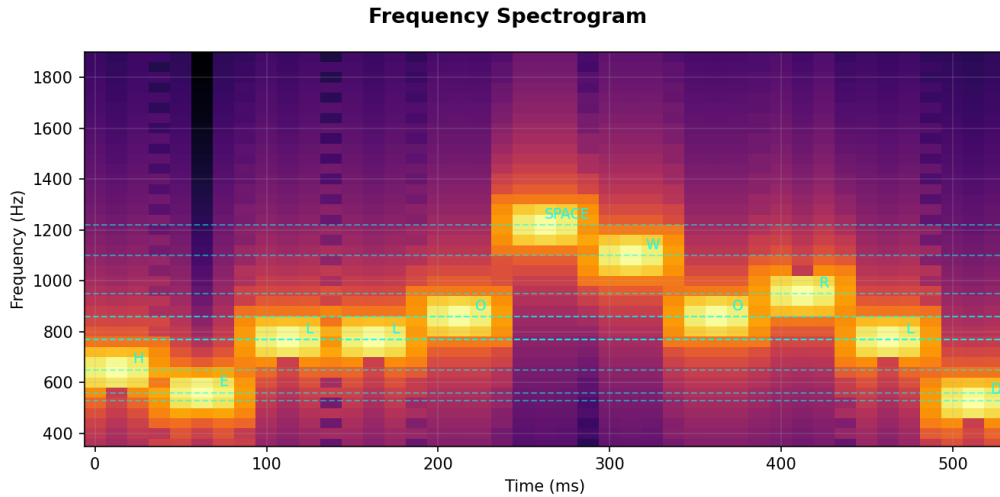


Figure 2: Simulated frequency spectrogram of the key sequence HELLO WORLD. Each cyan dashed line marks the expected frequency of a key.

9.3 Paragraph Input

In addition to the keypad, the application accepts a full paragraph in a multi-line text area. On **Analyze**, every valid keystroke in the text is processed through the same frequency pipeline:

1. The text is converted into a key sequence; supported keys are the 37 signatures (A–Z, 0–9, and SPACE).
2. Newlines and unsupported characters are skipped, exactly like any other invalid input.
3. Each key is processed through `generate_frequency` → `process_frequency`, producing a detected key, an error V , and a confidence C .
4. The reconstructed sequence and the per-key details are reported.

Because a paragraph can contain hundreds of keys, the waveform plots render only the first 30 keys (`MAX_PLOT_KEYS`) to stay responsive; the analysis and exported report always cover the full sequence.

9.4 Interactive Keypad and Live Updates

In addition to batch analysis, the application provides an on-screen **keypad** (A–Z + 0–9 + SPACE + Clear). Clicking a keypad button simulates a single key press and *incrementally updates* both plots live:

1. The key is appended to the accumulated sequence.
2. The frequency pipeline (`generate_frequency` → `process_frequency`) is executed for that key.

3. The spectrogram and the sine-wave plots are refreshed in place, growing with each added key.
4. The status bar shows the latest per-key frequency, detected key, error, and processing time.

This live interaction demonstrates the real-time nature of the system: every key event is processed immediately, within the deadline, and the reconstruction evolves continuously. The **Clear** button resets the sequence and the plots. The incremental update logic is factored into pure functions (`update_spectrogram`, `update_sine_plot`) so it is independently unit-testable.

9.5 Statistical Confidence

For every detected key, the system reports a per-key *confidence* score in the range 0.0–1.0 that expresses how close the detected frequency is to the predefined signature of the matched key:

$$C = 1 - \frac{|F_{\text{detected}} - F_{\text{expected}}|}{\text{TOLERANCE}},$$

clamped to $[0, 1]$. An error of 0 Hz gives $C = 1.0$ (a perfect match), and an error at the tolerance limit gives $C = 0.0$. The **UNKNOWN** state always reports $C = 0.0$. This provides a transparent, per-key measure of match quality alongside the raw error V .

9.6 Report Export and Copy

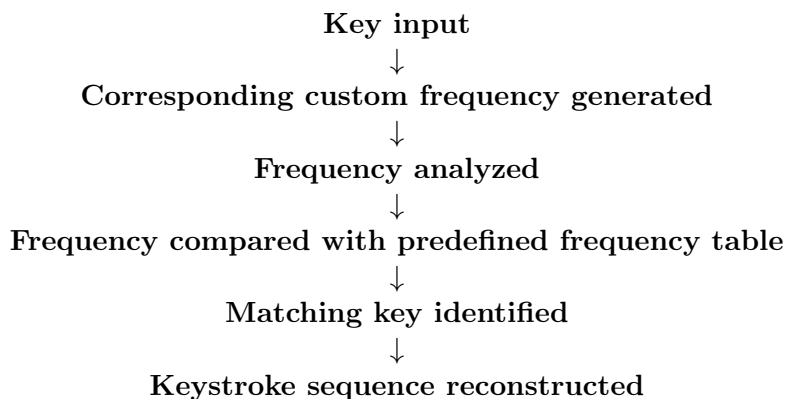
The complete analysis (invariant check, per-key frequency mapping with confidence, reconstructed sequence, WCET/deadline result, ranking, liveness and termination) is assembled by a single `format_report()` function and displayed in the output panel. From the **File** menu, the user can:

- **Export report:** save the analysis to a plain-text file, optionally including the sine and spectrogram plots as PNG images alongside it.
- **Copy results:** copy the current analysis output to the clipboard for pasting into another application.

Because both the on-screen output and the exported report are generated by the same `format_report()` function, the two are always identical.

10 Expected Outcome

The final software demonstrates:



10.1 Example

Input:	A	C	D	A
Frequency:	440	500	530	440 Hz
Analysis:	440→A	500→C	530→D	440→A
Reconstructed:	A	C	D	A

11 Frequency Database

Each key is assigned a unique frequency signature. The table below lists the assigned frequencies for all 48 keys: the 26 letters (A–Z), the 10 digits (0–9), the space bar, and 11 punctuation marks continued on the same 30 Hz grid.

Key	Frequency (Hz)	Key	Frequency (Hz)
A	440	N	830
B	470	O	860
C	500	P	890
D	530	Q	920
E	560	R	950
F	590	S	980
G	620	T	1010
H	650	U	1040
I	680	V	1070
J	710	W	1100
K	740	X	1130
L	770	Y	1160
M	800	Z	1190
SPACE	1220		
Key	Frequency (Hz)	Key	Frequency (Hz)
0	1250	5	1400
1	1280	6	1430
2	1310	7	1460
3	1340	8	1490
4	1370	9	1520
Key	Frequency (Hz)	Key	Frequency (Hz)
.	1550	,	1580
!	1610	?	1640
;	1670	:	1700
'	1730	"	1760
(	1790	)	1820
-	1850		

Table 1: Predefined frequency signatures for all 48 keys (26 letters, 10 digits, SPACE, and 11 punctuation marks). The signatures continue the 30 Hz grid above the digit keys.

12 Performance Results

Running the analysis on the sequence `HELLO WORLD` with synthetic noise enabled produces the following representative results. Each row shows the key’s assigned frequency, the detected key,

the frequency error V , and the associated confidence C :

Key	Frequency (Hz)	Detected	Error (Hz)	Conf.
H	648	H	2.3	0.71
E	562	E	2.4	0.70
L	774	L	3.8	0.53
L	773	L	2.6	0.67
O	861	O	1.0	0.88
SPACE	1224	SPACE	3.8	0.53
W	1097	W	2.7	0.66
O	857	O	3.4	0.58
R	946	R	3.6	0.55
L	765	L	4.9	0.39
D	534	D	3.8	0.53

Metric	Value	Requirement
Average Execution Time	< 0.1 ms	< 50 ms
WCET (median, 20 trials)	< 0.1 ms	< 50 ms
WCET (P95, 20 trials)	< 0.1 ms	< 50 ms
Average Error V	≈ 3.4 Hz	< 8 Hz (tolerance)
Confidence C	0.0–1.0	grows as $V \rightarrow 0$
Invariant Checks	All PASS	All True
Liveness	PASS	True
Termination	PASS	True

The multi-trial times are reported at microsecond scale, far below the 50 ms deadline, and every key is detected with a confidence proportional to how closely its detected frequency matches the predefined signature of the matched key.

13 Conclusion

This project successfully demonstrates a software-based simulation of an acoustic side-channel attack on a mechanical keyboard. By assigning unique frequency signatures to each key and implementing a matching/ranking mechanism, the system reconstructs keystroke sequences from the simulated frequency signals.

The implementation satisfies all the CPS verification requirements:

- **WCET** is measured across multiple trials and its median and P95 are shown to satisfy the real-time deadline.
- **Invariants** (positive, unique, and valid frequencies; sequence preservation) are defined and verified for all 48 keys.
- **Liveness** and **termination** are demonstrated programmatically.
- A **ranking function** $V = |F_{\text{detected}} - F_{\text{expected}}|$ quantifies match quality.
- A per-key **confidence** C expresses match quality in $[0, 1]$.
- **Safety** is ensured through the UNKNOWN state, which prevents incorrect key assignment.

The project provides a complete, verifiable CPS model and a graphical interface suitable for demonstration in a project presentation. The multi-line input box accepts a full paragraph and reconstructs every valid keystroke, including punctuation, while the on-screen keypad (A–Z + 0–9 + SPACE) lets a presenter type a sequence live and watch the sine and spectrogram plots grow with each key press, and the status bar reports the per-key frequency, error, and execution time in real time. The full analysis can be exported as a text report (optionally with the waveform plots) or copied to the clipboard for sharing.

14 Related Work

Real-world acoustic side-channel attacks operate on the broadband, transient sound of a keypress rather than a single pure tone. Harrison, Toreini, and Mehrnezhad classify laptop keystrokes with a deep-learning model (CoAtNet) on mel-spectrograms, reaching 95% accuracy for smartphone recordings and 93% over Zoom, and publicly release the recording corpus used ([JBFH-Dev/Keystroke-Datasets](#)); the dataset covers letters and digits only and provides raw audio, not per-key frequency constants. The `kbd-audio` toolkit by Gerganov (“KeyTap”) reconstructs keystrokes by building per-key acoustic fingerprints from a known transcript recorded on the target keyboard; it likewise requires live audio capture and offers no reusable frequency table.

These works motivate two design choices in this simulation: (a) every key is given a clearly separated, unique predefined signature (30 Hz spacing, 8 Hz tolerance) so the software-only model is unambiguous, and (b) punctuation keys are included so paragraph text reconstructs completely, even though no public acoustic dataset provides such mappings.